

CS3002E

Compiler Design

Run-Time Environments

Monsoon 2025

CSED NITC

Programming Language - Abstractions

- Abstractions - examples:
 - *Name, scope, binding, data type, operators, procedures*
- Compiler must accurately implement the *abstractions* in the source language.
 - e.g. *name* is an abstraction that gets mapped to a concrete store location

Run-Time Environment

Compiler creates and manages a *run-time environment* in which it assumes its target programs are being executed. This environment deals with issues like:

- allocation of storage locations for the objects named in the program
- mechanisms used by the target program to access variables
- mechanisms for parameter passing
- linkages between procedures
- interfaces to the operating system

Run-Time Environment

Specifically look at:

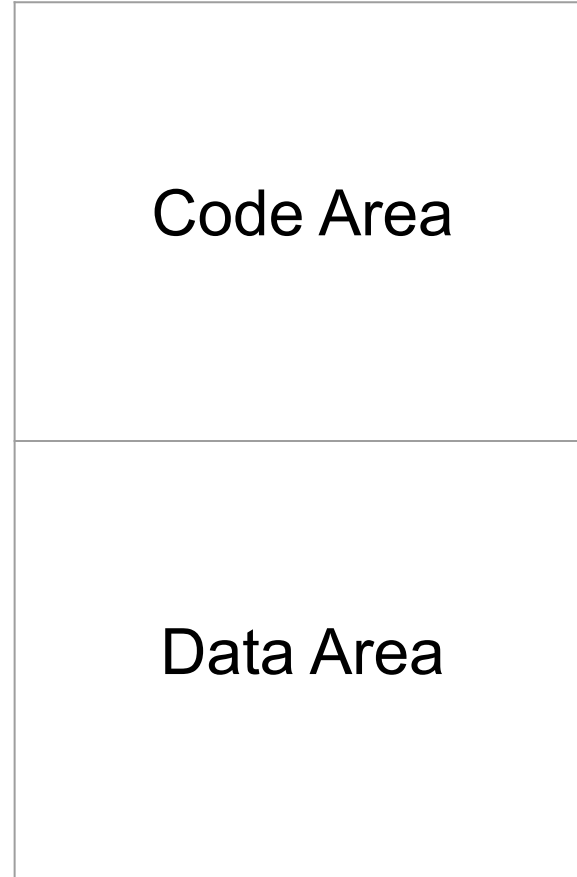
- allocation of storage locations
- access to variables
- implementing procedure calls

Storage Organization

- Compiler Writer's Perspective:
 - executing target program runs in its own logical address space
- Logical Address Space
 - Divided into *Code Area* and *Data Area*
- Logical Address Space is mapped by OS into physical address space

Run-time memory layout

Run-time memory divided into code area and data area



Code generation: mapping names to addresses

$z = x + y$ translates to (hypothetical machine architecture)

```
LD R1,  $x_{addr}$   
LD R2,  $y_{addr}$   
.....
```

x_{addr}, y_{addr} denote the address of variables x and y

Actual addresses of variables are not known during compilation.

Code generation: mapping names to addresses

Actual addresses of variables are not known during compilation.

But compiler knows the *relative address (offset)* of each variable

- available in the Symbol Table.

Simplifying assumptions

- No functions or procedures in the source language. A program consists of a single main function with declaration of local variables.
- All variables gets allocated space in the data area.
- Relative address for each variable (available in Symbol Table) is relative to the base of the data area.

Code generation: mapping names to addresses

```
LD R1, xaddr  
LD R2, yaddr  
.....
```

Maps each name to a relative address, relative to the *base* of the data area in memory

x at *base address + 0*

y at *base address + 4*

Is base address known during compilation?

Code generation: mapping names to addresses

Assuming register BP (Base Pointer) will contain the base address of the data area during execution, compiler can generate the following code:

```
LD R1, 0[BP]
```

```
LD R2, 4[BP]r
```

Source language with functions

Mapping names to addresses

```
int main() {int a; ... f(...) ...};  
...  
int f(int y) { int z; ... x=y+z ... }
```

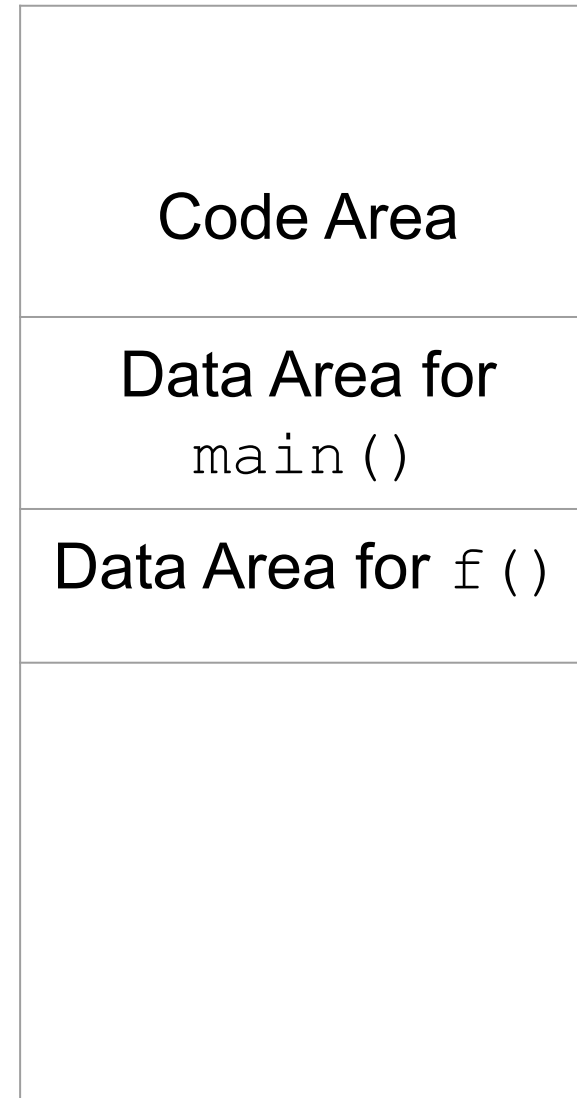
`x` is global

`y` is a parameter to function

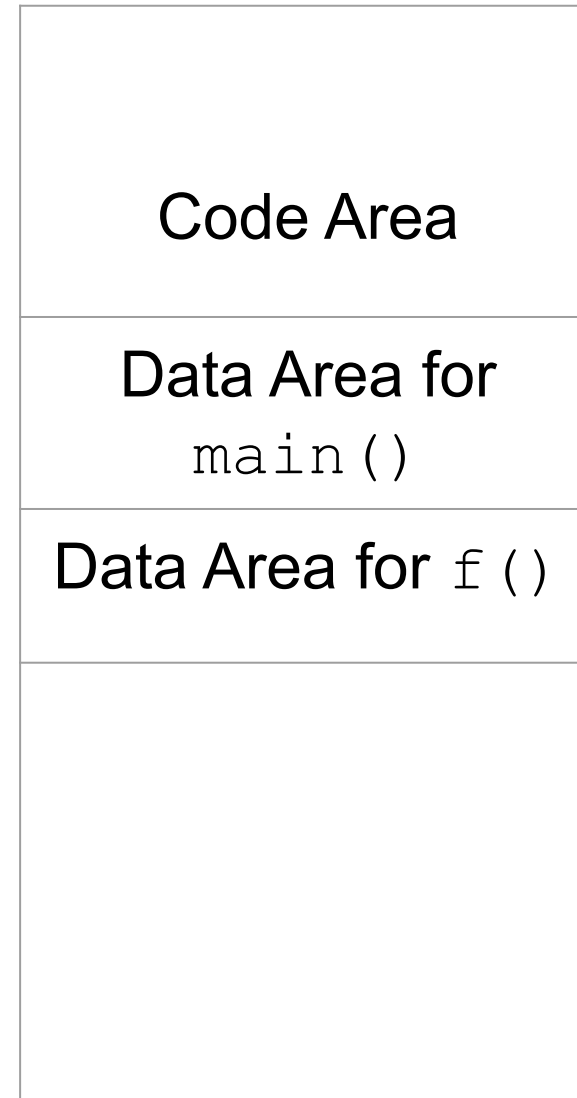
`z` is a local variable to a function

Run-time memory: Separate data area allocated for each function.

How is relative address computed for each name?



- Separate data area for each function.
- Mapping names to addresses
 - Address of a name that is local to a function is computed relative to the base of the data area for that function



What if $f()$ is recursive

```
int main() {int a; ... f(...) ...};  
...  
int f(int y) { int z; ... x=y+z ... f(...) ... }
```

Multiple activations of $f()$ at the same time.

`f ( )` is recursive

```
int main() {int a; ... f(...) ...};  
...  
int f(int y) { int z; ... x=y+z ... f(...) ... }
```

Multiple activations of `f ( )`.

Each activation requires separate space for storing its local variables, parameters etc.

`f ( )` is recursive

```
int main() {int a; ... f(...) ...};  
...  
int f(int y) { int z; ... x=y+z ... f(...) ... }
```

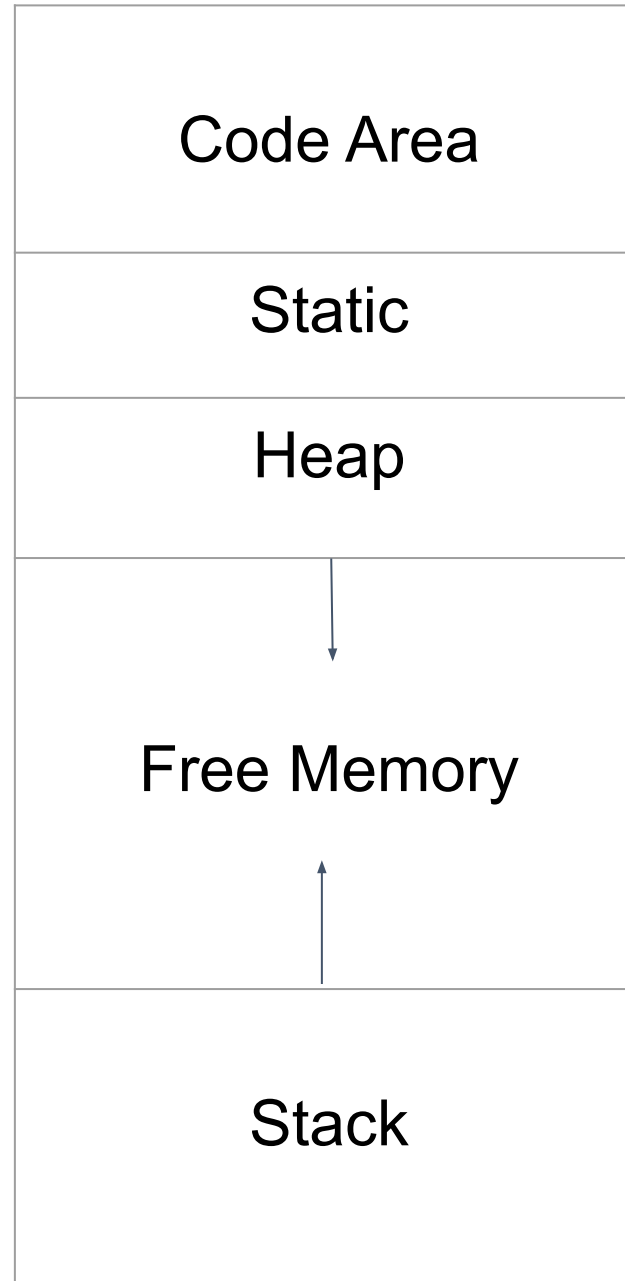
A separate data area required for each activation.

How many such data areas?

Number of activations is not known in advance.

Is the Base address of each data area known during compilation?

Run-time memory: Typical subdivision



Run-time Memory

Code Area - executable target program

Data Area

- Static Area - space for global constants, static variables etc.

- Heap Area - space for dynamically allocated data

- Stack Area - space for storing local variables, parameters and other data items of active functions

Heap Area grow in the opposite direction of stack area.

Life time

Life time - time during which memory is allocated for the variable

A data item can be local/global/static/dynamically allocated - decides its lifetime

- Global - lifetime is entire execution time of the program
- Local to a function - lifetime of an activation of the function
- Dynamic - from allocation to freeing of memory

Stack Allocation

To support procedures/functions

Each live activation of a procedure has an *activation record* in the stack

Activation Record (AR)

Activation Record (also known as **Frame**) - an area in memory allocated for storing data pertaining to an activation of a function.

- Storage in Stack Area (*Control Stack*).
- One Activation record for each activation of a function.
- The AR of the currently active function is on top of the stack.
- AR memory allocated upon call and released after returning.
- Multiple activations of the same function - a separate AR for each activation.

A general
activation record

Actual Parameters
Return value
Control link
Access link
Saved machine status
Local data
Temporaries

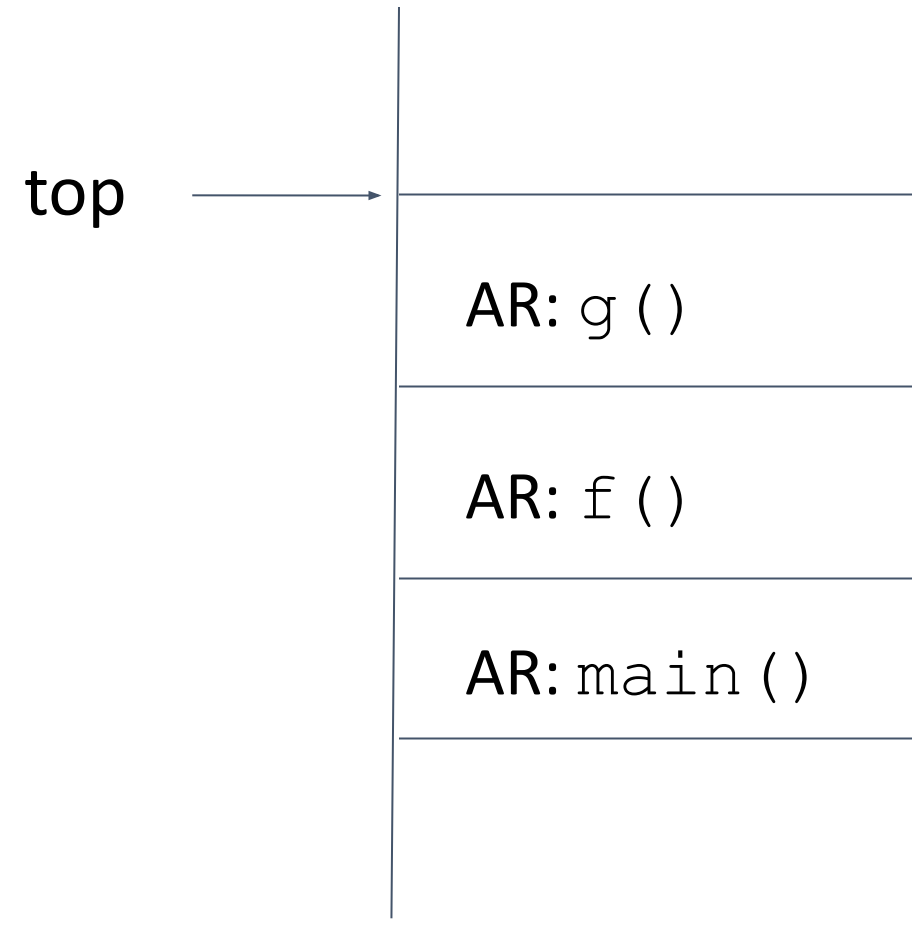
Activation Record (AR)

- Temporaries which can not be held in registers
- Local Data of the procedure that is active
- Saved machine status
 - information about the state of the machine just before the call - return address and contents of registers
- Space for return value
- Actual Parameters
- Control link pointing to the AR of the caller
- Access Link - to access non local data
 - for nested procedures - points to the AR of an enclosing procedure

Activation Record

Suppose `main()` calls `f()` and `f()` calls `g()`

How many Activation records in the stack when `g()` is active?



Caller / Callee

$f()$ calls $g()$

$f()$ is the *caller* and $g()$ is the *callee*.

Function Calls

Declaration / Prototype:

```
int g(int i, int j);
```

Calls / invocations - examples:

```
g(a, b)
```

```
g(a+b, x*y)
```

```
g(s[i], b+c)
```

```
g(g(a), h(b))
```

Function Call: Semantics

```
int g(int i, int j);
```

calls: $g(a, b)$

$g(a+b, x*y)$

$g(s[i], b+c)$

$g(g(a), h(b))$

Evaluate each argument, copy the value of the argument to the formal parameters i and j .

Space allocated for i and j in the AR of $g()$.

Functions: local variables

```
int g(int i, int j) {  
    int x, y;  
    . . . .  
};
```

Local variables x , y in AR of $g()$.

Function Calls: argument evaluation

Suppose $f()$ calls $g()$.

Who evaluates the arguments of $g()$?

Is it the job of the *caller* $f()$ or the *callee* $g()$?

Parameter Passing

$f()$ is calling $g()$:

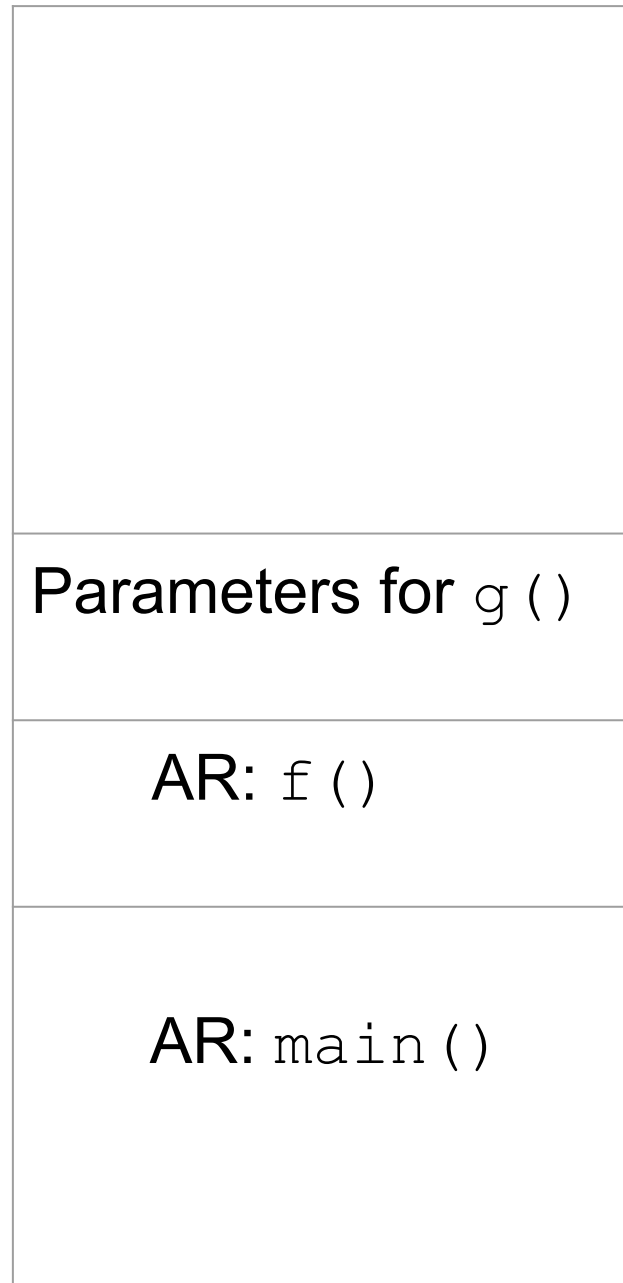
$f()$ is the current active function. Its AR is on top of the stack.

$f()$ evaluates arguments

$f()$ passes arguments to $g()$ - how?

`f ( )` starts setting up
AR for `g ( )` on top of
its own AR.

`f ( )` writes the
values of parameters
to the space just
above its AR



Function Calls

- While $g()$ is active, it stores its local variables and temporaries in its AR.
- When $g()$ finishes it should return value to $f()$, and control should return back to $f()$.

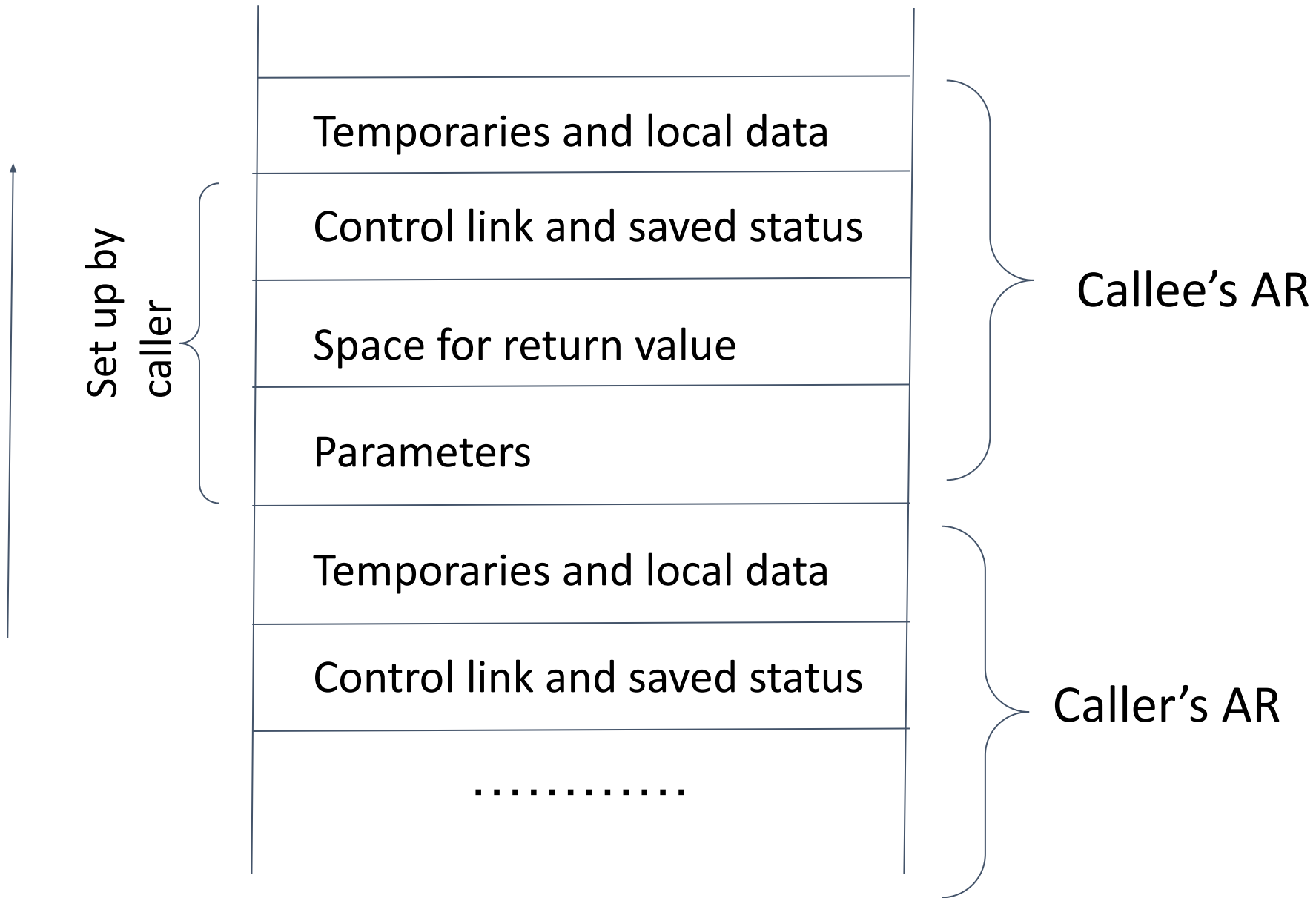
Where is the return value stored?

How $g()$ gets the return address ?

Calling Sequence

- Calling Sequence (also known as call sequence)
 - Code that allocates AR and enters information into it.
 - Divided between the caller and the callee.

It is desirable to put as much of the calling sequence into the callee as possible - why?



Layout of AR

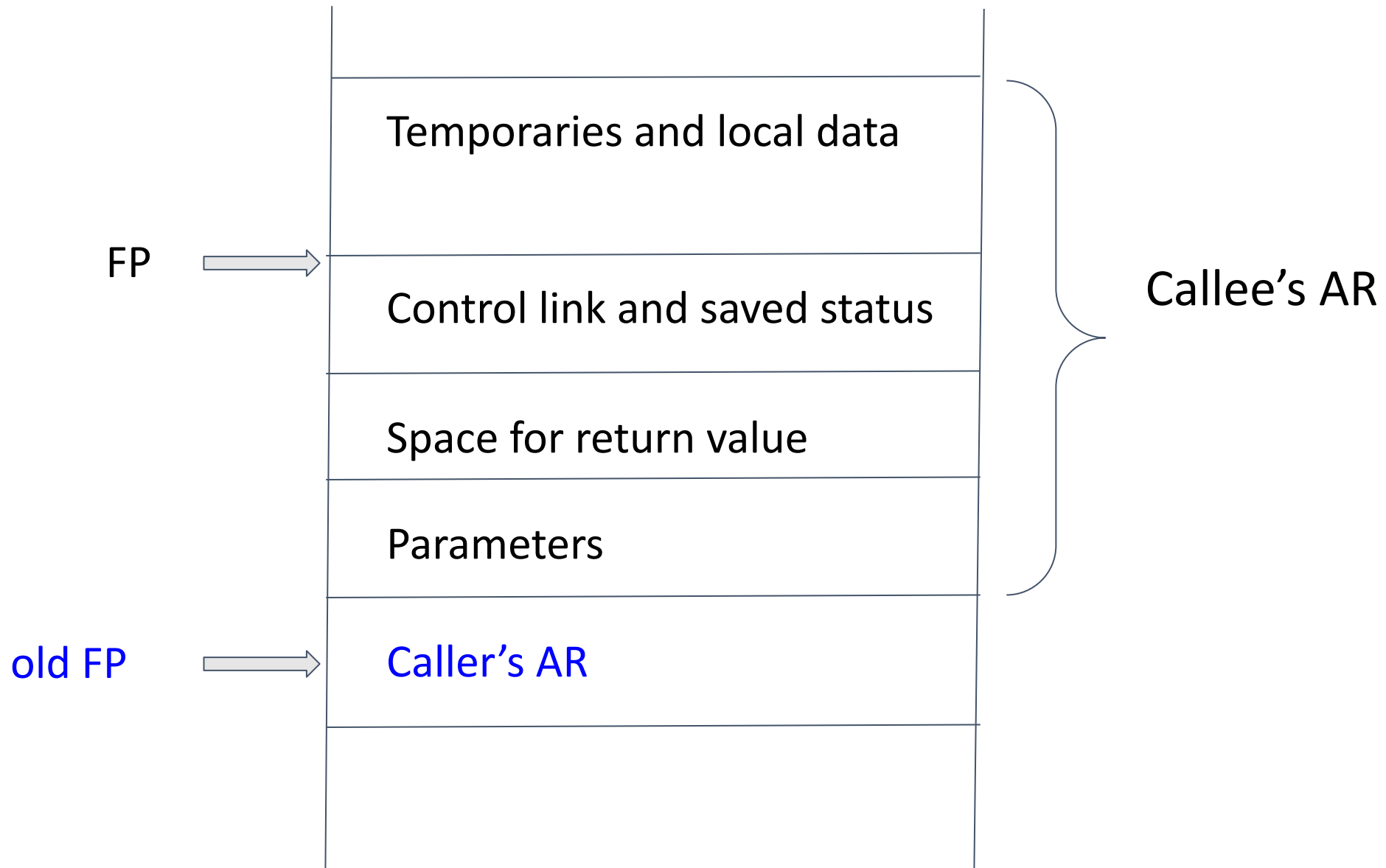
- Values communicated between the caller and the callee - at the beginning of the callee's AR
 - parameters, return value
- Fixed-length items in the middle
 - control Link, access link, machine status
- Local variables and temporaries at the end of the AR

Frame Pointer (FP)

- **Frame Pointer** (also known as **Base Pointer**) - a register that points to a fixed position in the current AR.
- Caller is responsible for setting up the value of FP before transferring control to the callee.
- Current value of FP (known as *Control link*) should be stored by the caller in the callee's AR.

Calling Sequence

1. The caller evaluates the actual parameters and writes into the callee's AR.
2. The caller stores a return address and the old value of FP into the callee's AR.
3. Caller increments value of FP to point to the position above the status field in callee's AR.
4. Callee saves register values and other status information.
5. Callee initializes its local data and begins execution.



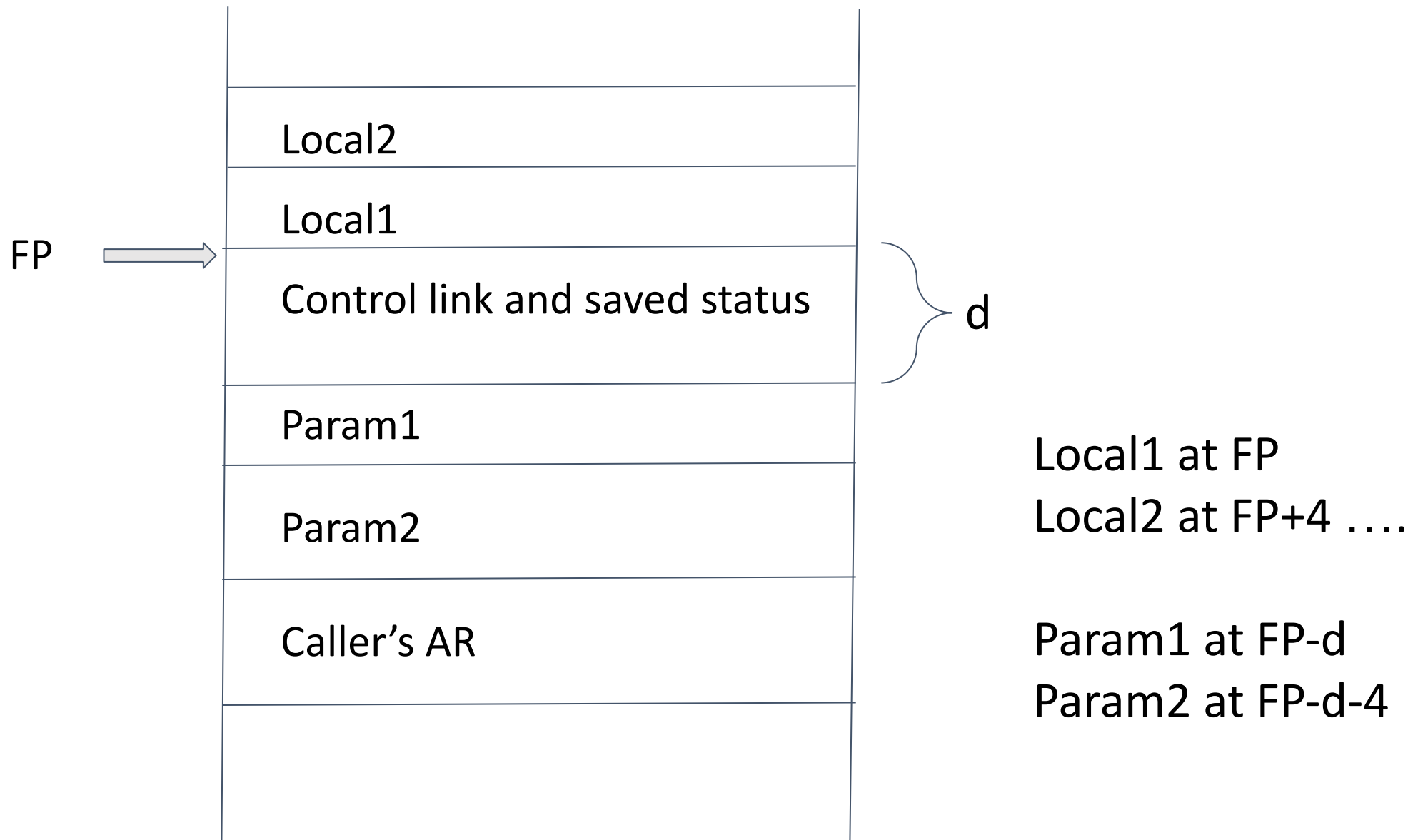
Return Sequence

1. Callee places the return value next to the parameters (in callee's AR)
2. Using the saved machine status information, callee restores FP and other registers, and then branches to the return address that the caller placed in the status field
3. The caller knows where the return value is, relative to the current value of FP, and can use it

Addresses of local data and parameters

The addresses of local data and parameters can be specified with respect to offsets from FP.

- Parameters at negative offsets and local data at positive offsets from FP.
- Code generator computes addresses as fixed offsets from FP.



Code generation: mapping names to addresses

source code:

```
int a=10,b=20;
```

code generated for a 64- bit, X86 processor:

```
movl    $10, -8(%rbp)
```

```
movl    $20, -4(%rbp)
```

`rbp` (base pointer register) points to the activation record of the current active function.

```
a=10;  
b=20;  
c=a+b;
```

```
movl    $10, -12(%rbp)  
movl    $20, -8(%rbp)  
movl    -12(%rbp), %edx  
movl    -8(%rbp), %eax  
addl    %edx, %eax  
movl    %eax, -4(%rbp)
```

a=10;

movl \$10, -12(%rbp)

b=20;

movl \$20, -8(%rbp)

c=a+b;

movl -12(%rbp), %edx

movl -8(%rbp), %eax

addl %edx, %eax

movl %eax, -4(%rbp)

```
int f() {  
    int fx=10;  
    return 0;  
}
```

```
int main(void) {  
    int mx=20;  
    ...  
    return 0;  
}
```

f:

```
...  
movl    $10, -4(%rbp)  
...
```

main:

```
...  
movl    $20, -4(%rbp)  
...
```


Architecture Dependent

Calling Conventions

Caller save / Callee save registers

Using registers to pass parameters / return values

Reference

ALSU Chapter 7